

Task-Based Asynchronous Programming in C#

In this article, I will discuss **Task-Based Asynchronous Programming in C#** with Examples. Please read our previous article, which discusses [How to Control the Result of a Task in C# using TaskCompletionSource](#) with Examples. In C#, the task is used to implement Asynchronous Programming, i.e., executing operations asynchronously, and it was introduced with .NET Framework 4.0. Before understanding theory, i.e., what is Task and the benefits of using Task, let us first discuss how to create and use Task in C#.

Working with Task in C#:

The Task-related classes belong to the [System.Threading.Tasks](#) namespace. So, the first and foremost step for you is to import the [System.Threading.Tasks](#) namespace in your program. Then, you can create and access the task objects using the Task class.

Note: In general, the Task class will always represent a single operation, and that operation will be executed asynchronously on a thread pool thread rather than synchronously on the application's main thread.

Example to Understand Task Class and Start Method in C#

We have already discussed `async` and `await` operators to create and execute the asynchronous methods. Now, let us try to understand how to implement asynchronous programming using the Task class. In the example below, we create the task object by using the Task class and then execute the method asynchronously by calling the Start method on the Task object. The method pointed by the Task object will be executed when we call the Start method.

```
using System;
using System.Threading;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}");
            Action actionDelegate = new Action(PrintCounter);
            Task task1 = new Task(actionDelegate);
            //You can directly pass the PrintCounter method as its signature instead of creating an
            Action delegate
            //Task task1 = new Task(PrintCounter);
            task1.Start();
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}");
            Console.ReadKey();
        }
        static void PrintCounter()
        {

```

```

        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId}
        Started");
        for (int count = 1; count <= 5; count++)
        {
            Console.WriteLine($"count value: {count}");
        }
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId}
        Completed");
    }
}
}

```

In the above example, we created the task object, i.e., task1, using the Task class and then called the Start method to start the task execution. Task object task1 will be executed asynchronously on a thread pool thread. Here, the Task class constructor expects one Action delegate. You can create an instance of the Action delegate and pass that action delegate instance as a parameter to the constructor, or you can directly pass a method whose signature is the same as the Action delegate. When you run the above application, you will get the following output.

```

Main Thread : 1 Started
Main Thread : 1 Completed
Child Thread : 3 Started
count value: 1
count value: 2
count value: 3
count value: 4
count value: 5
Child Thread : 3 Completed

```

As you can see in the above output, two threads are used to execute the application code. The main thread and the child thread. And you can observe both threads are running asynchronously.

Example to Understand How to Create a Task Object Using Factory Property in C#

In the previous example, the method will execute asynchronously when invoking the Start method. In the following example, we are creating the task object using the Factory property, which will start automatically, which means it will start executing the method immediately. Here, we don't need to call the Start method.

Here, the Factory property of the Task class will return an instance of the TaskFactory object. The TaskFactory class has one method called StartNew, which will require an Action delegate as a parameter. So, we can create an instance of Action delegate and pass that instance as a parameter to this StartNew method. Alternatively, you can directly pass a method matching the Action delegate signature. For a better understanding, please have a look at the following example.

```

using System;
using System.Threading;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{

```

```

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}");
        Task task1 = Task.Factory.StartNew(PrintCounter);
        Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
        Console.ReadKey();
    }
    static void PrintCounter()
    {
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Started");
        for (int count = 1; count <= 5; count++)
        {
            Console.WriteLine($"count value: {count}");
        }
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
    }
}

```

It will give you the same output as the previous example. The only difference between the previous and this example is that we create and run the task asynchronously using a single statement.

Example: Creating a Task Object using the Run method

In the following example, we create a task using the Run method of the Task class.

```

using System;
using System.Threading;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}");
            Task task1 = Task.Run(() => { PrintCounter(); });
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
            Console.ReadKey();
        }
        static void PrintCounter()
        {

```

```

        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId}
        Started");
        for (int count = 1; count <= 5; count++)
        {
            Console.WriteLine($"count value: {count}");
        }
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId}
        Completed");
    }
}
}

```

So, we have discussed three different ways to create and start a task in C#. From a performance point of view, **Task.Run** or **Task.Factory.StartNew** methods are preferable to create and start executing the tasks asynchronously. But, if you want the task creation and execution separately, you need to create the task separately by using the Task class and then call the Start method to start the task execution when required.

Task using Wait in C#:

As we already discussed, the tasks will run asynchronously on the thread pool thread, and the thread will start the task execution asynchronously along with the application's main thread. So far, the examples we discussed in this article, the child thread will continue its execution until it finishes its task, even after completing the main thread execution of the application.

If you want to make the main thread execution wait until all child tasks are completed, then you need to use the Wait method of the Task class. The Wait method of the Task class will block the execution of other threads until the assigned task has completed its execution. In the following example, we call the Wait() method on the task1 object to make the program execution wait until task1 completes its execution.

```

using System;
using System.Threading;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}
            Started");
            Task task1 = Task.Run(() =>
            {
                PrintCounter();
            });
            task1.Wait();
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}
            Completed");
            Console.ReadKey();
        }
        static void PrintCounter()
        {

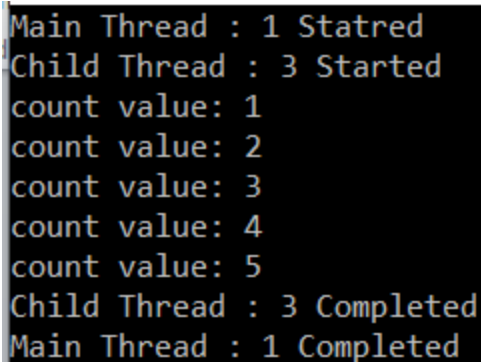
```

```

        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId}
        Started");
        for (int count = 1; count <= 5; count++)
        {
            Console.WriteLine($"count value: {count}");
        }
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId}
        Completed");
    }
}
}

```

As you can see in the above code, we are calling the Wait() method on the task object, i.e., task1. So, the main thread execution will wait until the task1 object completes its execution. Now run the application and see the output shown in the image below.



```

Main Thread : 1 Started
Child Thread : 3 Started
count value: 1
count value: 2
count value: 3
count value: 4
count value: 5
Child Thread : 3 Completed
Main Thread : 1 Completed

```

Task using Anonymous Method and Lambda Expression in C#:

In all our previous examples, we have executed a method using the Task. We have also seen that the Task class constructor, the Run method, or the StartNew method expect one Action delegate. So, instead of executing a method, we have also executed the logic using the Anonymous Method and Lambda Expression. For a better understanding, please have a look at the following example.

```

using System;
using System.Threading;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId}
            Started");
            #region Stat Method
            //Creating Task using Method
            Task task1 = new Task(PrintCounter);
            task1.Start();
            //Creating Task using Anonymous Method

```

```

Task task2 = new Task(delegate ()
{
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Started");
    Task.Delay(200);
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Completed");
});
task2.Start();
//Creating Task using Lambda Expression
Task task3 = new Task(() =>
{
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Started");
    Task.Delay(200);
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Completed");
});
task3.Start();
#endregion
#region StartNew
//Creating Task using Method
Task task4 = Task.Factory.StartNew(PrintCounter);
//Creating Task using Anonymous Method
Task task5 = Task.Factory.StartNew(delegate ()
{
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Started");
    Task.Delay(200);
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Completed");
});
//Creating Task using Lambda Expression
Task task6 = Task.Factory.StartNew(() =>
{
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Started");
    Task.Delay(200);
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Completed");
});
#endregion
#region Run
//Creating Task using Method
Task task7 = Task.Run(() => { PrintCounter(); });
//Creating Task using Anonymous Method
Task task8 = Task.Run(delegate ()
{
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.Manag
Started");
    Task.Delay(200);

```

```

        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
    });
    //Creating Task using Lambda Expression
    Task task9 = Task.Run(() =>
    {
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Started");

        Task.Delay(200);
        Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
    });
    #endregion

    Console.WriteLine($"Main Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
    Console.ReadKey();
}
static void PrintCounter()
{
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Started");
    Thread.Sleep(200);
    Console.WriteLine($"Child Thread : {Thread.CurrentThread.ManagedThreadId} Completed");
}
}
}

```

So, we have discussed how to work with tasks using different approaches. Now let us discuss what Task is and why we should use Task.

What Is a Task in C#?

A task in C# is used to implement Task-based Asynchronous Programming and was introduced with the .NET Framework 4. The Task object is typically executed asynchronously on a thread pool thread rather than synchronously on the application's main thread. A task scheduler is responsible for starting the Task and also responsible for managing it. By default, the Task scheduler uses threads from the thread pool to execute the Task.

The key type used in task-based asynchronous programming is Task and its generic counterpart Task<T>, where T is the result type. The Task class lives in the System.Threading.Tasks namespace and represents an asynchronous operation.

What is a Thread Pool in C#?

A thread pool in C# is a managed pool of threads created and managed by the .NET runtime to execute asynchronous tasks and parallel workloads efficiently. Thread pools are a fundamental component of the .NET Framework and are used to improve the efficiency and performance of multithreaded and asynchronous programming.

So, a **Thread Pool in C#** is a collection of **threads** that can perform several tasks in the background. Once a thread completes its task, it is returned back to the thread pool. This reusability of threads prevents an application from creating many threads, ultimately using less memory consumption.

Advantages of Task-Based Asynchronous Programming in C#:

Here are some key advantages of using TAP (Task-Based Asynchronous Programming):

Simplified Code Structure:

- **Readability:** Task-Based Asynchronous Programming allows for writing asynchronous code similar in structure to synchronous code, improving readability.
- **Maintainability:** The code is easier to maintain as it avoids the complexity of callbacks and manual thread management in older models.

Improved Scalability and Performance:

- **Efficient Resource Utilization:** Asynchronous operations free up the calling thread (typically UI or server threads) to handle other tasks. This improves resource utilization and responsiveness, particularly in UI applications or web services.
- **Scalability:** TAP can enhance the scalability of applications, especially those that handle many concurrent I/O-bound operations.

Language and Framework Support:

- **Language Integration:** Task-based asynchronous Programming is seamlessly integrated with C# language features, notably `async` and `await` keywords, making asynchronous programming more intuitive.
- **Framework Compatibility:** It's fully supported across the .NET ecosystem, including newer versions of .NET Core and .NET 5/6, ensuring compatibility and ease of use.

Exception Handling:

- **Simplified Exception Handling:** Exceptions in asynchronous methods can be caught and handled using standard try-catch blocks, unlike older patterns that require more complex handling.

Composability and Flexibility:

- **Composable Operations:** Tasks can be easily combined and composed. For instance, you can `await` multiple tasks concurrently using `Task.WhenAll`, or `await` the first task to complete using `Task.WhenAny`.
- **Cancellation Support:** Task-Based Asynchronous Programming supports cancellation using the `CancellationToken` class, allowing for responsive cancellation of asynchronous operations.

Unified Model for Asynchronous Operations:

- **Consistency:** TAP provides a unified approach for all asynchronous operations, whether CPU-bound or I/O-bound, creating consistency in how asynchronous code is written and understood across

different applications.

Progress Reporting:

- **Progress Feedback:** The model supports progress reporting out of the box, which is particularly useful in UI applications where you need to update the UI to reflect the progress of an asynchronous operation.

Disadvantages of Task-Based Asynchronous Programming in C#:

- **Complexity in Error Handling:** Asynchronous programming can make error handling more complex. Exceptions thrown in asynchronous methods are captured and placed on the Task, and they need to be handled using await or by examining the Task object. Unobserved exceptions can lead to unhandled exceptions.
- **Potential for Deadlocks:** Misusing async and await, especially with Task.Result or Task.Wait(), can lead to deadlocks, particularly in UI applications or when blocking on asynchronous code.
- **Resource Management:** Asynchronous operations can lead to more complex resource management scenarios. Ensuring that resources are properly disposed of or that certain operations are thread-safe adds complexity to the code.
- **Scalability Issues:** Asynchronous programming is great for scalability; improper use (like creating too many tasks or not using I/O-bound asynchronous APIs correctly) can consume many resources and degrade performance.
- **Debugging Difficulty:** Debugging asynchronous code can be more difficult than synchronous code. The execution flow is not linear, making it harder to follow and understand, especially when dealing with multiple concurrent asynchronous operations.
- **Overhead:** Some overhead is associated with managing the state and context of asynchronous operations. For very small operations, the overhead of setting up the asynchronous operation might outweigh its benefits.
- **Improper Usage:** It's easy to misuse asynchronous programming by applying it where it's not needed, leading to unnecessary complexity. Not every operation benefits from being made asynchronous.

In the next article, I will discuss [Chaining Tasks Using Continuation Tasks in C#](#) with Examples. Here, in this article, I try to explain Task-based Asynchronous Programming in C# using the Task class. I hope you understand how to create and using Task class objects in C#.